

LAPORAN PRAKTIKUM
STRUKTUR DATA
“laporan Praktikum Pekan 3”

Disusun Oleh:

Faiz Fikri Satria

2511533026

Dosen Pengampu : Dr. Wahyudi, S.T, M.T.
Asisten Praktikum : Sherly Sukmadira



DAPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa atas segala rahmat, taufik, serta hidayah-Nya yang melimpah, sehingga penulis dapat menyelesaikan laporan praktikum Struktur Data mengenai konsep dan implementasi "Stack" ini dengan tepat waktu dan tanpa hambatan yang berarti. Penulisan laporan ini merupakan bagian dari kewajiban akademik dalam menuntaskan mata kuliah Struktur Data di Departemen Informatika, Fakultas Teknologi Informasi, Universitas Andalas. Melalui serangkaian percobaan yang dilakukan di laboratorium, penulis mendapatkan pemahaman mendalam mengenai bagaimana data dikelola secara logis dalam memori komputer menggunakan prinsip-prinsip yang terstruktur.

Penulis ingin menyampaikan ucapan terima kasih yang sebesar-besarnya kepada Bapak Dr. Wahyudi, S.T, M.T. selaku dosen pengampu atas ilmu dan arahan yang diberikan selama perkuliahan. Terima kasih juga penulis sampaikan kepada Sherly Sukmadira selaku asisten praktikum yang telah membimbing selama sesi laboratorium, serta rekan-rekan mahasiswa yang telah bertukar pikiran selama proses penyelesaian tugas. Penulis menyadari bahwa laporan ini masih memiliki kekurangan, sehingga kritik dan saran yang membangun sangat diharapkan untuk perbaikan di masa yang akan datang. Semoga laporan ini dapat memberikan manfaat bagi pembaca dan menjadi referensi yang berguna dalam mempelajari ilmu komputer.

Padang 19 April, 2026

Penulis

DAFTAR ISI

KATA PENGANTAR	ii
DAFTAR ISI	iii
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan	1
1.3 Manfaat	2
BAB II PEMBAHASAN	3
2.1 Implementasi Stack Manual	3
2.1.1 Teori	3
2.1.2 Langkah-langkah	3
2.1.3 Contoh Kode	4
2.1.4 Analisis.....	4
2.2 Pencarian Data	4
2.2.1 Teori	4
2.2.2 Langkah-langkah	4
2.2.3 Contoh Kode	5
2.2.4 Hasil Output	5
2.2.5 Analisis	5
2.3 Stack Objek	5
2.2.1 Teori	5
2.2.2 Langkah-langkah	5

2.2.3 Contoh Kode	6
2.2.4 Hasil Output	6
2.2.5 Analisis	6
2.4 Evaluasi Postfix	6
2.2.1 Teori	6
2.2.2 Langkah-langkah	7
2.2.3 Contoh Kode	7
2.2.4 Hasil Output	7
2.2.5 Analisis	7
2.5 Tugas Pekan 2	7
2.5.1. Analisis	7
BAB III KESIMPULAN	9
DAFTAR PUSTAKA	10

BAB I

PENDAHULUAN

1.1 Latar Belakang

Struktur data linear merupakan elemen dasar dalam pengembangan perangkat lunak yang berfungsi untuk mengatur cara data disimpan dan diakses. Salah satu struktur data yang paling fundamental adalah Stack atau tumpukan, yang bekerja berdasarkan prinsip *Last In First Out* (LIFO). Dalam dunia informatika, Stack digunakan secara luas mulai dari manajemen memori (call stack), mekanisme *undo-redo* pada aplikasi pengolah kata, hingga proses navigasi pada peramban web. Pemahaman mengenai Stack sangat krusial karena hampir semua algoritma rekursif dan proses kompilasi kode bergantung pada mekanisme kerja tumpukan ini.

Melalui praktikum pekan ketiga ini, mahasiswa ditantang untuk mengimplementasikan Stack baik secara manual menggunakan array maupun menggunakan pustaka dinamis yang tersedia di Java. Hal ini bertujuan untuk memberikan perbandingan antara performa struktur data statis dan dinamis. Selain itu, penerapan Stack pada kasus evaluasi matematika seperti notasi postfix menjadi jembatan bagi mahasiswa untuk melihat bagaimana teori struktur data diaplikasikan dalam memecahkan masalah logika dunia nyata yang membutuhkan urutan operasi tertentu.

1.2 Tujuan

1. Memahami secara mendalam konsep operasi dasar Stack yaitu *push*, *pop*, *peek*, dan *isEmpty*.
2. Mampu mengimplementasikan Stack secara manual menggunakan array statis dengan penanganan kondisi *Overflow* dan *Underflow*.
3. Mampu mengelola Stack yang menampung objek kustom menggunakan bantuan ArrayList.
4. Mengimplementasikan algoritma untuk mencari nilai maksimum dalam tumpukan tanpa kehilangan data asli.

5. Menerapkan struktur data Stack dalam mengevaluasi ekspresi matematika notasi Postfix secara akurat.

1.3 Manfaat

1. Mahasiswa memiliki kemampuan untuk memilih jenis koleksi data yang tepat berdasarkan efisiensi memori dan waktu akses.
2. Meningkatkan keterampilan pemrograman Java, khususnya dalam hal manipulasi objek dan koleksi data.
3. Memberikan landasan logika yang kuat bagi mahasiswa untuk mempelajari struktur data yang lebih kompleks seperti Tree dan Graph di masa mendatang.

BAB II

PEMBAHASAN

2.1 Implementasi Stack Manual

2.1.1 Teori

Stack adalah struktur data linear yang menerapkan prinsip *Last In First Out* (LIFO), di mana elemen yang terakhir dimasukkan akan menjadi yang pertama dikeluarkan. Implementasi menggunakan array memerlukan variabel penunjuk top untuk melacak indeks elemen teratas.

2.1.2 Langkah-langkah

1. Mendeklarasikan array dengan ukuran tetap (MAX).
2. Membuat fungsi push untuk menambah elemen setelah memeriksa apakah stack penuh (*Overflow*).
3. Membuat fungsi pop untuk mengambil elemen teratas setelah memeriksa apakah stack kosong (*Underflow*).

2.1.3 Contoh Kode

```
1 package pekan2_2511533026;
2 import java.util.ArrayList;
3
4 public class ArrayList1_2511533026 {
5     public static void main(String[] args) {
6         //Size of the ArrayList
7         int n = 5;
8         // Declaring the ArrayList with initial size n
9         ArrayList<Integer> arrli = new ArrayList<Integer>(n);
10        for (int i = 1; i <= n; i++)
11            arrli.add(i);
12        //Printing elements
13        System.out.println(arrli);
14        //Remove element at index 3
15        arrli.remove(3);
16        //Displaying the ArrayList
17        //after deletion
18        System.out.println(arrli);
19        //Printing elements one by one
20        for (int i = 0; i < arrli.size(); i++)
21            System.out.print(arrli.get(i) + " ");
22    }
23 }
```

2.1.4 Analisis

ini sangat efisien dalam penggunaan memori statis karena alokasi dilakukan di awal. Namun, kelemahannya adalah fleksibilitas; jika jumlah data melebihi MAX, diperlukan pembuatan array baru atau penanganan error.

2.2 Pencarian Data

2.2.1 Teori

Operasi pencarian pada Stack tidak bisa dilakukan secara langsung (seperti akses indeks pada array) karena hanya elemen teratas yang terlihat. Oleh karena itu, diperlukan struktur bantuan untuk menyimpan elemen sementara.

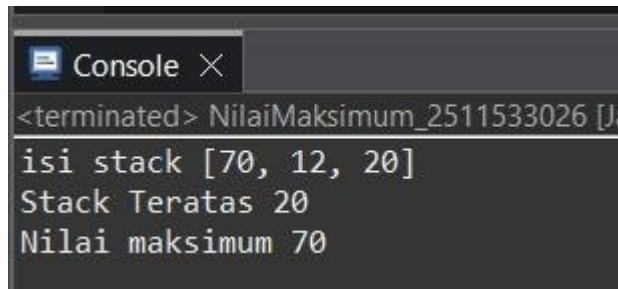
2.2.2 Langkah-langkah

1. Menyiapkan satu stack utama dan satu stack pembantu.
2. Memindahkan isi stack utama ke pembantu sambil membandingkan setiap nilai dengan variabel max.
3. Mengembalikan seluruh elemen dari stack pembantu ke stack utama agar struktur data tidak rusak.

2.2.3 Contoh Kode

```
1 package pekan3_2511533026;  
2 import java.util.Stack;  
3  
4 public class NilaiMaksimum_2511533026 {  
5     public static int max(Stack<Integer> s) {  
6         Stack<Integer> backup = new Stack<Integer>();  
7         int maxVal = s.pop();  
8         backup.push(maxVal);  
9         while (!s.isEmpty()) {  
10            int next = s.pop();  
11            backup.push(next);  
12            maxVal = Math.max(maxVal, next);  
13        }  
14        while (!backup.isEmpty()) {  
15            s.push(backup.pop());  
16        }  
17        return maxVal;  
18    }  
19    public static void main (String[] args) {  
20        Stack<Integer> s = new Stack<Integer>();  
21        s.push(70);  
22        s.push(12);  
23        s.push(20);  
24        System.out.println("Isi stack "+s);  
25        System.out.println("Stack teratas "+s.peek());  
26        System.out.println("Nilai maksimum "+max(s));  
27    }  
28 }
```


2.2.4 Hasil Output



```
<terminated> NilaiMaksimum_2511533026 [Ja
isi stack [70, 12, 20]
Stack Teratas 20
Nilai maksimum 70
```

2.2.4 Analisis

Proses ini memiliki kompleksitas waktu $O(n)$ karena setiap elemen harus diperiksa satu per satu. Penggunaan stack pembantu krusial agar data tetap utuh setelah proses pencarian selesai.

2.3 Stack Objek

2.3.1 Teori

Stack tidak hanya terbatas pada tipe data primitif, tetapi juga dapat menyimpan referensi objek. Dengan menggunakan ArrayList sebagai dasarnya, kita mendapatkan keuntungan berupa ukuran yang dinamis.

2.3.2 Langkah-langkah

1. Membuat kelas model Siswa dengan atribut NIM dan Nama.
2. Menginisialisasi ArrayList<Siswa> di dalam kelas pembungkus Stack.
3. Mengarahkan operasi push ke method add() dan pop ke method remove(size-1).

2.3.3 Contoh Kode

```
1 package petani_2511533026;
2
3 public class StackArray_2511533026 {
4     static final int MAX_SIZE = 1000;
5     int top;
6     int a[MAX_SIZE];
7     boolean isEmpty() {
8         return (top < 0);
9     }
10    StackArray_2511533026() {
11        top = -1;
12    }
13    boolean push (int x) {
14        if (top >= (MAX_SIZE - 1)) {
15            System.out.println("Stack Overflow");
16            return false;
17        }
18        else {
19            a[++top] = x;
20            System.out.println(x + " dimasukkan dalam stack");
21            return true;
22        }
23    }
24    int pop() {
25        if (top < 0) {
26            System.out.println("Stack Underflow");
27            return 0;
28        }
29        else {
30            int x = a[top--];
31            return x;
32        }
33    }
34 }
```

2.2.4 Hasil Output

```
Console
<terminated> SiswaStack_2511533026 [Java Application] C:\Users\Lenovo\p2\
Siswa teratas: Nim: 3, Nama: Charles
Mengeluarkan siswa teratas: Nim: 3, Nama: Charles
Daftar siswa setelah di-pop:
Nim: 2, Nama: Bobby
Nim: 1, Nama: Ali
```

2.3.4 Analisis

Penggunaan ArrayList menghilangkan risiko *Stack Overflow* secara logis karena memori akan bertambah secara dinamis. Ini adalah pendekatan yang lebih modern dalam pemrograman Java.

2.4 Evaluasi Postfix

2.4.1 Teori

Notasi postfix (Reverse Polish Notation) menghilangkan kebutuhan akan tanda kurung dalam operasi matematika. Stack digunakan untuk menyimpan operand hingga ditemukan operator yang sesuai.

2.4.2 Langkah-langkah

1. Membaca ekspresi dari kiri ke kanan.
2. Masukkan angka ke dalam Stack.
3. Jika bertemu operator, ambil dua angka teratas, hitung, dan masukkan kembali hasilnya ke Stack.

2.4.3 Contoh Kode

```
1 package pekan3.2511533026;  
2 import java.util.Scanner;  
3  
4 public class StackPostfix_2511533026 {  
5     public static int postfixEvaluate(String expression) {  
6         Stack<Integer> s = new Stack<Integer>();  
7         Scanner input = new Scanner(expression);  
8         while (input.hasNextInt()) { // an operand (integer)  
9             if (input.hasNextInt()) {  
10                s.push(input.nextInt());  
11            }  
12            } else { // an operator  
13                String operator = input.next();  
14                int operand2 = s.pop();  
15                int operand1 = s.pop();  
16                if (operator.equals("+")) {  
17                    s.push(operand1 + operand2);  
18                } else if (operator.equals("-")) {  
19                    s.push(operand1 - operand2);  
20                } else if (operator.equals("*")) {  
21                    s.push(operand1 * operand2);  
22                } else {  
23                    s.push(operand1 / operand2);  
24                }  
25            }  
26            input.close();  
27        }  
28        return s.pop();  
29    }  
30  
31 public static void main(String[] args) {  
32     System.out.println("hasil postfix = " + postfixEvaluate("5 2 4 * 7 -"));  
33 }  
34 }
```

2.4.4 Analisis

Algoritma ini menunjukkan kekuatan Stack dalam menangani presedensi operasi secara otomatis. Hal ini merupakan dasar dari cara kerja kalkulator dan *compiler*.

2.5 Tugas Pekan 2

2.5.1 Analisis

Berdasarkan hasil eksekusi program, manipulasi data pada ArrayList terbukti jauh lebih sederhana dibandingkan array manual. Analisis hasil menunjukkan bahwa metode `sort()` bekerja secara efektif dengan kompleksitas $O(n \log n)$. Program mampu memberikan informasi statistik (min, max, rata-rata) secara instan segera setelah input selesai diberikan, membuktikan efisiensi penggunaan memori dinamis untuk pengolahan data koleksi.

```

31
32 case 2: // Tombol Back (Pop)
33     if (history.isEmpty()) {
34         System.out.println("History kosong! Tidak ada halaman yang bisa di-back.");
35     } else {
36         website_2511533026.dihapus = history.pop();
37         System.out.println("Halaman \"" + dihapus.getJudul_3026() + "\" berhasil di-back!");
38     }
39     break;
40
41 case 3: // Lihat Halaman Saat Ini (Peek)
42     if (history.isEmpty()) {
43         System.out.println("Tidak ada halaman yang sedang aktif.");
44     } else {
45         website_2511533026.aktif = history.peak();
46         System.out.println("Halaman Saat Ini:");
47         System.out.println("Judul : " + aktif.getJudul_3026());
48         System.out.println("URL : " + aktif.getUrl_3026());
49     }
50
51 // Cek Status History
52 System.out.println("Stack Status History:");
53 System.out.println("Total riwayat = " + history.size());
54 System.out.println("Stack kosong? = " + history.isEmpty());
55 break;
56
57 case 4: // Keluar
58     System.out.println("Terima kasih telah menggunakan Simulasi Browser History!");
59     sc.close();
60     System.exit(0);
61     break;
62
63 default:
64     System.out.println("Pilihan tidak valid! Silakan pilih 1-4.");
65 }
66 }
67 }
68 }
69 }

```

```

1 package Pekan3_2511533026;
2 import java.util.Scanner;
3
4 public class Browser_2511533026 {
5     public static void main(String[] args) {
6         StackWebsite_2511533026 history = new Stack<>();
7         Scanner sc = new Scanner(System.in);
8
9         System.out.println("=== Browser History NIM: 2511533026 ===");
10
11         while (true) {
12             System.out.println("\n1. Kunjungi Website (Push)");
13             System.out.println("2. Tombol Back (Pop)");
14             System.out.println("3. Lihat Halaman Saat Ini (Peek)");
15             System.out.println("4. Keluar");
16             System.out.print("Pilihan: ");
17
18             String pilihanStr = sc.nextLine().trim();
19             int pilihan = Integer.parseInt(pilihanStr);
20
21             switch (pilihan) {
22                 case 1: // Kunjungi Website (Push)
23                     System.out.print("Masukkan Judul: ");
24                     String judul = sc.nextLine().trim();
25                     System.out.print("Masukkan URL: ");
26                     String url = sc.nextLine().trim();
27
28                     Website_2511533026 websiteBaru = new Website_2511533026(judul, url);
29                     history.push(websiteBaru);
30                     System.out.println("Berhasil mengunjungi halaman!");
31                     break;
32             }
33         }
34     }
35 }

```

```

1 package Pekan3_2511533026;
2
3 public class Website_2511533026 {
4     private String judul_3026;
5     private String url_3026;
6
7     // Constructor
8     public Website_2511533026(String judul_3026, String url_3026) {
9         this.judul_3026 = judul_3026;
10        this.url_3026 = url_3026;
11    }
12
13    // Getter (Selektor)
14    public String getJudul_3026() {
15        return judul_3026;
16    }
17
18    public String getUrl_3026() {
19        return url_3026;
20    }
21
22    // Setter (Mutator)
23    public void setJudul_3026(String judul_3026) {
24        this.judul_3026 = judul_3026;
25    }
26
27    public void setUrl_3026(String url_3026) {
28        this.url_3026 = url_3026;
29    }
30 }
31 }

```

Console X

Browser_2511533026 [Java Application] D:\eclipse\plugins\org.eclipse.justj.

=== Browser History NIM: 2511533026 ===

1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Saat Ini (Peek)
4. Keluar

Pilihan:

BAB III

KESIMPULAN

Praktikum pekan ketiga ini memberikan pemahaman yang komprehensif mengenai struktur data Stack, baik dari sisi implementasi internal (array vs dinamis) maupun aplikasi praktisnya. Melalui percobaan pembuatan Stack manual, penulis memahami mekanisme top dan batasan memori statis. Sementara itu, penggunaan ArrayList menunjukkan kemudahan dalam mengelola objek kompleks secara fleksibel. Salah satu poin penting yang dipelajari adalah aplikasi Stack dalam evaluasi postfix yang mendemonstrasikan bagaimana struktur data ini mempermudah penyelesaian masalah logika matematika yang berurutan.

Selain itu, integrasi materi Tugas Pekan 2 memperkuat pemahaman mengenai pentingnya pemilihan struktur data yang tepat. Sementara Stack sangat unggul untuk operasi LIFO, ArrayList memberikan keunggulan dalam hal akses acak dan pengurutan data. Secara keseluruhan, penguasaan terhadap kedua konsep ini sangat krusial dalam membangun aplikasi yang efisien, di mana integritas data tetap terjaga melalui metode *push-pop* pada Stack maupun manipulasi dinamis pada koleksi Java.

DAFTAR PUSTAKA

1. Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2014). *Data Structures and Algorithms in Java*. Wiley.
2. Oracle. (2023). *The Java™ Tutorials – Lesson: Introduction to Collections*.
3. Sedgewick, R., & Wayne, K. (2011). *Algorithms*. Addison-Wesley.